

PE01

SWT301 FALL24 - Final Practical Exam

Duration: 85 Minutes

Instructions:

- You are expected to demonstrate proficiency in at least one programming language relevant to the questions presented.
 - You may use IDEs such as NetBeans, IntelliJ IDEA, or other similar software during the examination.
 - Your completed exam work must be submitted as a single Microsoft Word document. No other file formats will be accepted for grading.
-

Question 1: Code Defects (3 Points)

The following Java class (provided below) contains six identifiable defects related to code standards, logic errors, and best practices. Analyze the code carefully and identify each defect. Use the provided Defect Report Template below to document your findings.

```
public class fileProcessor {
    private BufferedReader reader;
    private String filePath;

    public fileProcessor (String filePath) {
        try {
            reader = new BufferedReader (new FileReader(filePath));
            if (filePath == null || filePath.isEmpty()) {
                System.out.println("File opened successfully!");
            }
        } catch (IOException e) {
            System.out.println("Error opening file: " + e.getMessage());
        }
    }

    public void readFile() {
        String line;
        try {
```

```

        while ((line = reader.readLine()) != null) {
            System.out.println(line);
        }
    } catch (IOException e) {
        System.out.println("Error reading file: " + e.getMessage());
    } finally {
        reader.close();
    }
}

public void processFile(){
    openFile(filePath);
    System.out.println("Processing file...");
    if(filePath == null){
        System.out.println("Invalid file path");
        return;
    }

    readFile();
}
}

```

Defect Report Template:

For each identified defect, complete the following information:

- **Defect ID:** [Unique Identifier for the defect, e.g., DF001, DF002, etc.]
- **Defect Name:** [Descriptive name for the defect, such as "Naming Convention Error" or "Logic Error"]
- **Line Number:** [Specific line number in the code where the defect occurs]
- **Defect Description:** [A brief, one-sentence explanation of the issue]
- **Fixing Solution:** [Concise description of the code change to resolve the defect]

Question 2: JUnit Testing (5 Points)

The following Java method (provided below) calculates the total price for a customer's order based on item prices, customer type, discount codes, and VIP status. The method employs different calculations for various discount scenarios.

Your task is to design a comprehensive set of JUnit test cases for this method using both equivalence partitioning and boundary value analysis techniques. Ensure your test cases provide 100% code coverage, meaning that all branches of the code are executed by your tests.

```
public class OrderCalculator {
    public double calculateTotalPrice(double itemPrice, String customerType,
    boolean isVIP, String discountCode) {
        if (itemPrice <= 0) {
            throw new IllegalArgumentException("Item price must be
positive.");
        }
        if (customerType == null || customerType.length() == 0) {
            throw new IllegalArgumentException("Customer type is invalid.");
        }
        double totalPrice = itemPrice;
        double discount = 0.0;

        if(isVIP) {
            discount = 0.1; // 10% discount for VIP Customers
        } else if (customerType.equalsIgnoreCase("student")) {
            discount = 0.05; // 5% discount for student customers
        } else if (customerType.equalsIgnoreCase("senior")) {
            discount = 0.02; // 2% discount for senior customers
        }

        if(discountCode != null) {
            if(discountCode.equalsIgnoreCase("SAVE10")) {
                discount += 0.1; // Additional 10% discount for SAVE10 code
            } else if(discountCode.equalsIgnoreCase("WELCOME5")) {
                discount += 0.05; // Additional 5% discount for WELCOME5 code
            }
        }

        double finalPrice = totalPrice * (1 - discount);
        return finalPrice; // Price does not go below zero.
    }
}
```

Deliverables:

1. **JUnit Test Class:** Create a JUnit test class for the `OrderCalculator` method. Include the necessary test cases to achieve full equivalence partitioning, boundary value analysis, and branch coverage.
2. **Test Case Summary:** For each test case, provide a one-line summary using the following format:

ID: <Test Case ID>, Test for: <Brief Description of Test Goal>, Input parameter: <Input Values Used>, Expected result: <Anticipated Output>

Example:

ID: TC1, Test for: Negative item price, Input parameter: -1, "Regular", true, "SAVE10", Expected result: `IllegalArgumentException` is thrown

3. **Test Code:** Include the Java code for each test case in your submission.

Example Test Case (Illustrative):

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class OrderCalculatorTest {

    @Test
    public void testNegativeItemPrice() {
        assertThrows(IllegalArgumentException.class, () -> {
            OrderCalculator calculator = new OrderCalculator();
            calculator.calculateTotalPrice(-1, "Regular", true, "SAVE10");
        });
    }
}
```

Question 3: Use Case Testing (2 Points)

You are given the following use case for an online shopping system. Your task is to create functional black-box test cases based on this use case using use-case-based testing techniques.

Use Case: Online Shopping System - Place an Order

- **Use Case Name:** Place an Order
- **Primary Actor:** Customer
- **Preconditions:**
 - The customer must be logged into the system.
 - The shopping cart must have at least one item.
- **Main Success Scenario:**
 1. The customer adds items to their shopping cart.
 2. The customer applies a discount code (optional).
 3. The system validates the discount code.
 4. The customer proceeds to checkout.
 5. The system displays the total amount, including discounts.
 6. The customer provides payment information.
 7. The system processes the payment.
 8. The system confirms the order and generates an order confirmation.
- **Alternate Scenarios:**
 - A1: If the shopping cart is empty when proceeding to checkout, the system prompts the customer to add items to the cart.
 - A2: If the discount code is invalid, the system displays an error message and allows the customer to proceed without applying the discount.
 - A3: If the payment fails (e.g., due to insufficient funds or incorrect card details), the system notifies the customer and prompts them to retry payment.
- **Postconditions:**
 - The customer receives an order confirmation with a summary of the items and the final total (including discounts and taxes).
 - The system updates the inventory based on the items ordered.
- **Products:** Product A: \$50, Product B: \$30, Product C: \$20
- **Valid Discount Codes:** SAVE10: 10% off, WELCOME5: 5% off for new customers.
- **Payment Methods:** Credit/Debit Cards, PayPal.
- **Payment Failures:** Insufficient funds or invalid card details.

Instructions:

Based on the provided use case, create functional black-box test cases to validate the system's behavior. Ensure that your test cases cover:

1. The normal flow of the use case.
2. The alternative flows described.
3. The exception scenarios outlined.

For each test case, specify the following:

- **Test Case ID:** (e.g., UC001, UC002, etc.)
- **Test Case Description:** (A brief description of what the test case aims to verify)
- **Preconditions:** (Conditions that must be met before the test can be executed)
- **Test Steps:** (Detailed sequence of actions, including test data)
- **Expected Results:** (The anticipated outcome of the test steps)
- **Notes:** (Specify if the test case covers Normal Flow (NF), Alternative Flow (AL), or Exception (EX))

Example Test Case:

Test Case 1: Normal Flow - Successful Order with Standard Delivery

- **ID:** TC001
 - **Description:** Tests a successful order placement with standard delivery using a valid credit card and no discount code.
 - **Preconditions:**
 - The customer is logged in to the system.
 - The shopping cart contains at least one item (e.g., a tea cup for \$2).
 - **Test Steps:**
 1. Customer adds Product A to their cart.
 2. Customer proceeds to checkout.
 3. Customer checks the total amount displayed.
 4. Customer selects credit card payment.
 5. Customer enters valid credit card details (e.g., valid card number, expiry date, CVV).
 6. The system processes the payment.
 7. The system confirms the order and generates an order confirmation.
 - **Expected Results:**
 - The customer receives an order confirmation with the order details and an estimated delivery time.
 - The system updates the inventory based on the ordered product.
 - **Notes:** Normal Flow (NF)
-

Important Notes:

- Remember to clearly document your test cases with descriptive IDs, descriptions, preconditions, test steps, and expected results.
- Pay close attention to boundary conditions and equivalence partitions when creating your test cases for Question 2.
- When constructing test cases, feel free to state any reasonable assumptions needed to clarify your answers.

PE02

SWT301 FALL24 - Final Practical Exam (Retake)

Duration: 85 Minutes

Instructions:

- You are expected to demonstrate proficiency in at least one programming language relevant to the questions presented.
 - You may use IDEs such as NetBeans, IntelliJ IDEA, or other similar software during the examination.
 - Your completed exam work must be submitted as a single Microsoft Word document. No other file formats will be accepted for grading.
-

Question 1: Code Defects (3 Points)

The following Java class (provided below) contains six identifiable defects related to code standards, logic errors, and best practices. Analyze the code carefully and identify each defect. Use the provided Defect Report Template below to document your findings.

```
public class EmployeeManager {

    private List<String> employeeNames = new ArrayList<>();
    private Connection dbConnection;

    public void ConnectToDatabase(String url){
        try {
            dbConnection = DriverManager.getConnection(url);

            if (dbConnection == null || url.isEmpty()) {
                System.out.println("Connected to the database.");
            }
        }
    }
}
```

```

        } catch (SQLException e) {
            System.out.println("Error connecting to database: " +
e.getMessage());
        }
    }

    public void addEmployee(String name) {
        if (name.length() > 0){
            employeeNames.add(name);
        } else {
            return;
            System.out.println("Employee name cannot be empty.");
        }
    }

    public void getAllEmployees(){
        if(dbConnection == null){
            System.out.println("Database connection is not established.");
            return;
        }

        try{
            Statement stat = dbConnection.createStatement();
            ResultSet rs = stat.executeQuery("SELECT * FROM employees");

            while (rs.next()) {
                System.out.println(rs.getString("name"));
            }

            rs.close();

        } catch (SQLException e) {
            System.out.println("Error fetching employees: " + e.getMessage());
        }
    }

    public void closeConnection(){
        dbConnection.close();
    }

```



```
}  
}
```

Defect Report Template:

For each identified defect, complete the following information:

- **Defect ID:** [Unique Identifier for the defect, e.g., DF001, DF002, etc.]
 - **Defect Name:** [Descriptive name for the defect, such as "Naming Convention Error" or "Logic Error"]
 - **Line Number:** [Specific line number in the code where the defect occurs]
 - **Defect Description:** [A brief, one-sentence explanation of the issue]
 - **Fixing Solution:** [Concise description of the code change to resolve the defect]
-

Question 2: JUnit Testing (5 Points)

The following Java method calculates the bonus for an employee based on years of service, salary, and performance. The method involves different calculations for various scenarios.

Your task is to design test cases using JUnit for this method. You should apply equivalence partitioning and boundary value analysis for the input parameters. Additionally, ensure that your test cases cover all branches of the code to achieve 100% code coverage.

```
public class EmployeeBonusCalculator {  
  
    public double calculateBonus(int yearsOfService, double salary, boolean  
hasOutstandingPerformance) {  
        if (yearsOfService <= 0 || salary <= 0) {  
            throw new IllegalArgumentException("Invalid input: yearsOfService  
and salary must be positive.");  
        }  
  
        double bonusPercentage = 0.0;  
  
        if (yearsOfService > 10) {  
            bonusPercentage = 0.10;  
        } else if (yearsOfService >= 5 ) {  
            bonusPercentage = 0.07;  
        } else if (yearsOfService >= 1) {  
            bonusPercentage = 0.05;  
        }  
    }  
}
```

```

        } else {
            bonusPercentage = 0.02; // Should not reach here due to input
validation
        }

        if (hasOutstandingPerformance) {
            bonusPercentage += 0.03;
        }

        return salary * bonusPercentage;
    }
}

```

Sample JUnit Test Class for Completion:

The following is a partially completed JUnit test class for the `EmployeeBonusCalculator` method. Students are required to complete this class by adding the necessary test cases to achieve full equivalence partitioning, boundary value analysis, and branch coverage for the `EmployeeBonusCalculator` method. After completing the tests, students should copy and paste the test code into a Word document for evaluation.

```

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class EmployeeBonusCalculatorTest {

    @Test
    public void testInvalidYearsOfService() {
        assertThrows(IllegalArgumentException.class, () -> {
            EmployeeBonusCalculator calculator = new
EmployeeBonusCalculator();
            calculator.calculateBonus(-1, 50000, true);
        });
    }

    @Test
    public void testYearsOfServiceGreaterThanOrEqualToTen() {
        EmployeeBonusCalculator calculator = new EmployeeBonusCalculator();
        double bonus = calculator.calculateBonus(11, 50000, false);
        assertEquals(5000, bonus, 0.01); // $5000 should be 10% of 50000
    }
}

```

```
}  
  
    // Add more code to complete the test suite  
}
```

After completing the tests, students should provide a one-line summary for each test case and copy the test code into a Word document for evaluation.

Example:

ID: TC1; Test for: Invalid years of service; Input parameter: -1, 50000, true; Expected result: IllegalArgumentException

Test code:

```
@Test  
public void testInvalidYearsOfService() {  
    // implementation here  
}
```

Question 3: Use Case Testing (2 Points)

You are given the following use case for a Hotel Reservation system. This system enables customers to perform the following functions: select a room type, create a reservation, and make a payment.

Your task is to create functional black-box test cases based on this use case using use-case-based testing techniques.

Use Case: Make a Hotel Reservation

- **Primary Actor:** Customer
- **Preconditions:**
 - The customer must have an account on the hotel booking system.
 - The hotel must have available rooms for the selected dates.
- **Main Success Scenario (Normal Flow):**
 1. The customer logs into the system.
 2. The customer selects the dates for check-in and check-out.
 3. The system shows available rooms for the selected dates.
 4. The customer selects a room type (e.g., Single, Double, Suite).

5. The customer provides personal information (e.g., name, contact details) and payment information.
 6. The system verifies the payment details and confirms the reservation.
 7. The system sends a confirmation email with the reservation details.
- **Alternative Flows:**
 - A1: If the customer enters invalid dates (e.g., check-out date before check-in date), the system displays an error and asks for new dates.
 - A2: If no rooms are available for the selected dates, the system offers alternative dates or room types.
 - A3: If the payment is declined (e.g., insufficient funds or expired card), the system prompts the customer to provide a different payment method.
 - A4: If the customer cancels the reservation process before completing payment, the system cancels the process without saving any details.
 - **Exception Scenarios:**
 - E1: If the system experiences an error while processing payment, it displays an error message and logs the user out.
 - E2: If the customer tries to reserve a room after the check-in date has already passed, the system prevents the reservation and prompts the customer to select new dates.
 - **Postconditions:**
 - The customer receives a confirmation email with reservation details (if the booking is successful).
 - The system updates the availability of the rooms for the selected dates.

Based on the use case above, create functional black-box test cases to validate the system's behavior. Ensure that your test cases cover:

1. The normal flow.
2. The alternative flows.
3. The exception scenarios.

For each test case, specify:

- **Test Case ID:**
- **Test Case Description:**
- **Preconditions:**
- **Test steps (Test Procedure) with customized test data:**
- **Expected results:**
- **Notes: (NF, AL or EX)**

Ensure that each test case tests a unique aspect of the use case to demonstrate a comprehensive understanding of use-case-based testing.

Test Case Example:

Test Case 1: Normal Flow - Successful Reservation

- **ID:** TC001
 - **Description:** Tests a successful reservation through the system.
 - **Preconditions:**
 - The customer has an account and is logged in.
 - The hotel has available rooms for the selected dates.
 - **Test Steps:**
 1. The customer selects check-in and check-out dates.
 2. The customer chooses a room type (e.g., Double).
 3. The customer enters personal information and valid payment details.
 4. The system processes the payment and confirms the reservation.
 - **Expected Results:**
 - The system confirms the reservation.
 - A confirmation email is sent to the customer with reservation details.
 - **Notes:** Normal flow (NF)
-

Important Notes:

- Remember to clearly document your test cases with descriptive IDs, descriptions, preconditions, test steps, and expected results.
- Pay close attention to boundary conditions and equivalence partitions when creating your test cases for Question 2.
- When constructing test cases, feel free to state any reasonable assumptions needed to clarify your answers and make the test more accurate.